# Python Engineering - Class 03

Bucles e iteracion | Guía de ejercicios

**Objetivo de la clase:** Aprender cuando un problema requiere repeticion, como controlar una iteracion y como utilizarla para construir algoritmos numericos.

Este documento es la guía base del ejercicio. No contiene una solución completa para copiar: organiza el problema, los pasos de implementación, los valores de referencia y la validación.

## Ruta

```
01_python_bootcamp/
`-- week_01_logic/class_03_loops/
    |-- notes/
    `-- exercises/
```

## Flujo de trabajo obligatorio

```
PROBLEM
INPUTS
OUTPUTS
EQUATIONS
SPECIAL CASES
DO I NEED ITERATION?
ALGORITHM
VALIDATION
```

Antes de programar, redacta estas secciones. El objetivo es separar el razonamiento de la sintaxis.

## EX05 - Suma de cuadrados

**Problema:** Calcular S=sum(i^2) desde i=1 hasta N mediante acumulacion.

**Entradas de referencia:** N entero positivo

**Ecuaciones / relación:** S = sum(i**2), i=1..N; S=N(N+1)(2N+1)/6

**Comprobación esperada:** Para N=3 el resultado es 14. Comparar la iteración con la fórmula analítica.

### Guía de implementación

- 1. Inicializar total=0.
- 2. Recorrer range(1,N+1).
- 3. Acumular i**2.
- 4. Comparar con formula cerrada.

### Antes de ejecutar

Predice el signo, el orden de magnitud, el tipo de dato y cualquier shape relevante. Después compara esa predicción con la salida real.

## EX06 - Factorial

**Problema:** Calcular n! con un bucle ascendente y otro descendente.

**Entradas de referencia:** n entero no negativo

**Ecuaciones / relación:** n! = n*(n-1)*...*1; 0! = 1

**Comprobación esperada:** Para n=5 el resultado es 120. Comparar con math.factorial().

### Guía de implementación

- 1. Inicializar producto=1.
- 2. Recorrer factores.

- 3. Multiplicar acumulador.
- 4. Validar con math.factorial().

### Antes de ejecutar

Predice el signo, el orden de magnitud, el tipo de dato y cualquier shape relevante. Después compara esa predicción con la salida real.

## EX07 - Serie de Taylor de exp(x)

**Problema:** Aproximar e^x mediante una suma finita de terminos x^n/n!.

**Entradas de referencia:** N=5; xp=0.5

**Ecuaciones / relación:** exp(x) aproximada por sum(x**n/n!), n=0..N

**Comprobación esperada:** Comparar con math.exp(0.5). Al aumentar N cerca del punto de expansión debería reducirse el error de truncamiento.

### Guía de implementación

- 1. Incluir termino n=0.
- 2. Mantener factorial incremental.
- 3. Acumular terminos.
- 4. Evaluar en xp.
- 5. Comparar con exp().

### Antes de ejecutar

Predice el signo, el orden de magnitud, el tipo de dato y cualquier shape relevante. Después compara esa predicción con la salida real.

## EX08 - Movimiento vertical con Euler

**Problema:** Integrar dh/dt=v y dv/dt=-g mediante Euler explicito hasta alcanzar el suelo.

**Entradas de referencia:** h0=0; v0=50; t0=0; g=9.81; dt=0.1

**Ecuaciones / relación:** Euler: h_new=h+v*dt; v_new=v-g*dt; t_new=t+dt

**Comprobación esperada:** h_max analítica aproximadamente 127.42 m; tiempo de vuelo aproximadamente 10.19 s; el resultado numérico depende de dt y de la regla de parada.

### Guía de implementación

- 1. Validar dt,g y condiciones iniciales.
- 2. Inicializar estado.
- 3. Iterar mientras h>=0.
- 4. Actualizar h,v,t.
- 5. Guardar maximo.
- 6. Comparar con solucion analitica.

### Antes de ejecutar

Predice el signo, el orden de magnitud, el tipo de dato y cualquier shape relevante. Después compara esa predicción con la salida real.

## Validación final

Responde estas preguntas como comentarios al final del ejercicio. El objetivo es demostrar que puedes explicar el flujo del código.

- Diferencia entre for y while.
- Por que una suma empieza con acumulador 0 y un producto con 1?
- Que significa que range() excluye stop?
- Que representa N en la serie de Taylor?
- Que es dt en Euler y que ocurre al reducirlo?
- Por que una simulacion necesita una condicion de parada?

## Criterio de cierre

La clase se considera completada cuando el código se ejecuta, las comprobaciones de referencia tienen sentido, puedes explicar cada bloque sin leer una solución y puedes modificar al menos un input anticipando su efecto.